SYRIAN ARAB REPUBLIC
SYRIAN PRIVATE UNIVERSITY
FACULTY OF SYSTEM SECURITY & COMPUTER NETWORK

Cloud IDS
(Intrusion Detection Systems)

Prepared by:

Nawar Daher

Obeida Fattouh

Mohhamad Hakmeh

Supervised by:

Dr. Wasim Junaidi

Eng. Ahmad Shekha

Table of Contents

Contents

Abstract:

Intrusion Detection Systems (IDSes) are extensively used to detect potential threats within guest virtual machines (VMs). However, conventional IDSes often struggle to fulfill the stringent needs of high-performance cloud environments. Firstly, the process of monitoring VM events tends to increase the tail latency experienced by guest services. Secondly, the throughput capacity of traditional IDS solutions is insufficient for the high volume of events generated in such clouds, resulting in missed detections and compromised accuracy. Lastly, many cloud providers implement sophisticated IDS tools directly inside the VMs, which makes updating and maintaining these systems challenging, as any new functionality or improvements necessitate modifications to the guest VMs themselves

Currently, the importance of Intrusion Detection Systems (IDS) has grown significantly, especially with the increasing variety and sophistication of cyber threats targeting cloud infrastructure and digital services. Detecting attacks such as intrusions, malware, and persistent advanced threats has become essential to ensure data security and system integrity. Additionally, international regulations and standards, such as data protection and privacy laws, require organizations to implement stringent measures for early threat detection and effective response. Therefore, developing IDS that can adapt to high-performance cloud environments and provide a high level of security and efficiency is a critical factor for ensuring business continuity and safeguarding vital assets in today's digital age.

تُستخدم أنظمة كشف التسلل (IDS) على نطاق واسع للكشف عن التهديدات المحتملة داخل الأجهزة الافتراضية الضيفة. ومع ذلك، غالباً ما تواجه أنظمة كشف التسلل التقليدية صعوبة في تلبية المتطلبات الصارمة لبيئات الحوسبة السحابية عالية الأداء. أولًا، تميل عملية مراقبة أحداث الأجهزة الافتراضية إلى زيادة زمن الاستجابة الذي تعاني منه الخدمات الضيفة. ثانيًا، لا تكفي سعة معالجة البيانات لحلول أنظمة كشف التسلل التقليدية للتعامل مع الحجم الكبير للأحداث المتولدة في هذه البيئات السحابية، مما يؤدي إلى عدم اكتشاف بعض الأحداث وانخفاض دقة الكشف. أخيرًا، يقوم العديد من مزودي خدمات الحوسبة السحابية بتطبيق أدوات كشف التسلل المتطورة مباشرةً داخل الأجهزة الافتراضية، مما يجعل تحديث هذه الأنظمة وصيانتها أمرًا صعباً، حيث تتطلب أي وظائف جديدة أو تحسينات إجراء تعديلات على الأجهزة الافتراضية الضيفة نفسها.

كما تزايدت أهمية أنظمة كشف التسلل (IDS) بشكل ملحوظ في الوقت الراهن، لا سيما مع تزايد تنوع وتعقيد التهديدات السيبرانية التي تستهدف البنية التحتية السحابية والخدمات الرقمية. وأصبح كشف الهجمات، كالتسلل والبرمجيات الخبيثة والتهديدات المتقدمة المستمرة، ضروريًا لضمان أمن البيانات وسلامة النظام. إضافةً إلى ذلك، تتطلب اللوائح والمعايير الدولية، كقوانين حماية البيانات والخصوصية، من المؤسسات تطبيق إجراءات صارمة للكشف المبكر عن التهديدات والاستجابة الفعّالة لها. لذا، يُعدّ تطوير أنظمة كشف التسلل القادرة على التكيف مع بيئات الحوسبة السحابية عالية الأداء، وتوفير مستوى عالٍ من الأمان والكفاءة، عاملاً حاسمًا لضمان استمرارية الأعمال وحماية الأصول الحيوية في عصرنا الرقمي.

CHAPTER 1
introduction

1.1 Introduction:

In today's world, where reliance on cloud computing is continually increasing, organizations depend heavily on cloud infrastructure to store data and run applications and services. As dependence on the cloud grows, new challenges related to information security and sensitive data protection have emerged, with cyber-attacks evolving constantly to target cloud environments specifically. Consequently, there is a growing need for advanced and effective systems to monitor and detect threats and attacks in cloud environments quickly and reliably. This is where the Cloud Intrusion Detection System (Cloud IDS) project comes into play, offering a modern solution to address these challenges and enhance security levels within cloud computing environments.

1.2 problem definition:

This project faces several fundamental issues related to traditional detection systems: as they are unable to meet the demands of high-performance data processing. This is due to the limited processing rate of these traditional systems, which does not satisfy the fast and accurate detection requirements in high-performance environments. As a result, important events may be missed, and the overall detection accuracy is reduced, negatively impacting the system's effectiveness.

1.3 Project goal:

i) Study and design of intrusion detection systems (IDS) architectures and their operational mechanisms in cloud environments, with the aim of understanding and analyzing their capabilities in detecting and responding to security threats within these environments.

ii) Simulation of a cloud environment at the virtual level to evaluate and study issues related to processing large data volumes and requests, as well as analyzing challenges associated with event loss in intrusion detection systems, in order to improve system performance and increase accuracy in threat detection.

1.4Reference study

Tab.1

Title	Journal	Pub Year	Authors	Achievement	Benefit in project
EIDS:A cloud Intrusion Detection system with High performance and maintainability	*ACM Transaction on computer systems*	*2026*	*Xiaokang Hu Zhichao Hua Nainan Guan Yan Xu Yibin shen Yang Yu zeyu mi Yubin Xia Ming Wang Jiesheng wu*	*. A new ids frame design called Eids . Divide all data into two sections . Using Micro VM . scheduler development*	*The Paper provides. practical and high performance architecture for cloud ids systems it directly aligns with our project goal by addressing the Key challenges of latency throughput, and maintainability in modern cloud environments*
Intrusion Detection Systems in Cloud Computing Paradigm: Analysis and Overview	*Complexity*	*2022*	*-Pooja Rana -Isha Batra -Arun Malik -Agbotiname Lucky Imoize - Yongsung Kim - Subhendu Kumar Pani - Nitin Goyal - Arun Kumar - Seungmin Rho*	*. Analysed Intrusion Detection Systems (IDS) in cloud computing environments . Compared four IDS techniques: FCM-ANN, SVM-ANN, FCM-SVM, and SMO-ANN . Used two benchmark datasets: NSL-KDD and UNSW-NB15 . Evaluated performance using metrics such as Accuracy, Precision, Detection Rate, detecting attacks*	*Understanding IDS in cloud environments Identifying limitations of traditional IDS Using machine learning techniques for attack detection Applying performance evaluation metrics.*
Towards an Intelligent Intrusion Detection System to Detect Malicious Activities in Cloud Computing	*Journal of Computer virology and hacking techniques*	*2023*	*Azidine Guezza Mouaad Mohy-Eddine Hanane Attou Sara Benkirane Mohamed Azrour Abdulrahman Alabdulatif Nouf Almuslem*	*A Novel Hybrid IDS Model High Accuracy with Minimal Features Effectively Addressing Data Imbalance Improved Detection of Normal Traffic Enhanced Computational Efficiency*	*This paper propose a cloud intrusion detection System designed. to overcome the Limitation of traditional IDS deployed inside Virtual machines, by moving intrusion detection function outside gust Vm The proposal system Significantly performance and maintainability*

Adversarial Threats to Cloud IDS: Robust Defence With Adversarial Training and Feature Selection	*IEEE Access*	*2025*	*Hariprasad Holla Shashidhar Reddy Polepalli Arun Ambika Sasikumar*	*. Proof that these attacks significantly reduce detection accuracy Increased detection accuracy to approximately 88% after protection .*	*Through this, we can understand the weaknesses of IDS in cloud computing learn how to improve detection accuracy use machine learning in detection and develop robust IDS against attacks.*
Utilizing IDS and IPS to Improve Cybersecurity Monitoring Process	*Journal of Cyber Security and Risk Auditing*	*2025*	*Mony Ho Sopheaktra Huy Midhunchakkaravarthy Janarthanan Sokroeurn Ang*	*The difference between IPS &IDS Detection methods IDS problems The importance of integrating IPS with IDS*	*This can be used to explain the differences between IDS and IDS to explain IDS problems support the project idea.*
Building a Cloud-IDS by Hybrid Bio Inspired Feature Selection Algorithms Along With Random Forest Model	*IEEE Access*	*2024*	*Mhamad Bakro Rakesh Ranjan Kumar Mohammad Husain Zubair Ashraf Arshad Ali Syed Irfan Yaqoob Mohammad Nadeem Ahmed Nikhat Parveen*	*Developed a Cloud Intrusion Detection System (Cloud-IDS) using hybrid feature selection (GA + GOA) and Random Forest to improve attack detection accuracy.*	*Helps in designing an efficient Cloud Intrusion Detection System and improving attack detection using machine learning techniques.*

CHAPTER 2

Reference study

2.1 intro:

This section provides a theoretical overview of the relationship between cloud concepts (IaaS, PaaS, and SaaS), the principles of Internet Survey Definition Systems (ISS), and the use of the Falco system in a cloud environment. It also covers the concept of a secure cloud environment, its fundamentals, and its security. The discussion will explore how service model selection aligns with security and choice requirements, and how these elements impact system design, development costs, and operational expenses.

2.2 Cloud Definition

According to NIST's definition, the cloud is defined as a model for providing computing resources (such as servers, storage, and applications) on demand through technical interfaces, taking into account key characteristics: scalability, service-by-service management, and deployment flexibility (public, private, hybrid), in addition to specific security and compliance frameworks.

Many organizations and researchers have defined the architecture for cloud computing. Basically the whole system can be divided into the core stack and the management. In the core stack, there are three layers: (1) Resource (2) Platform and (3) Application. The resource layer is the infrastructure layer which is composed of physical and virtualized computing, storage and networking resources.

The platform layer is the most complex part which could be divided into many sub-layers. a computing framework manages the transaction dispatching and/or task scheduling. A storage sub-layer provides unlimited storage and caching capability.

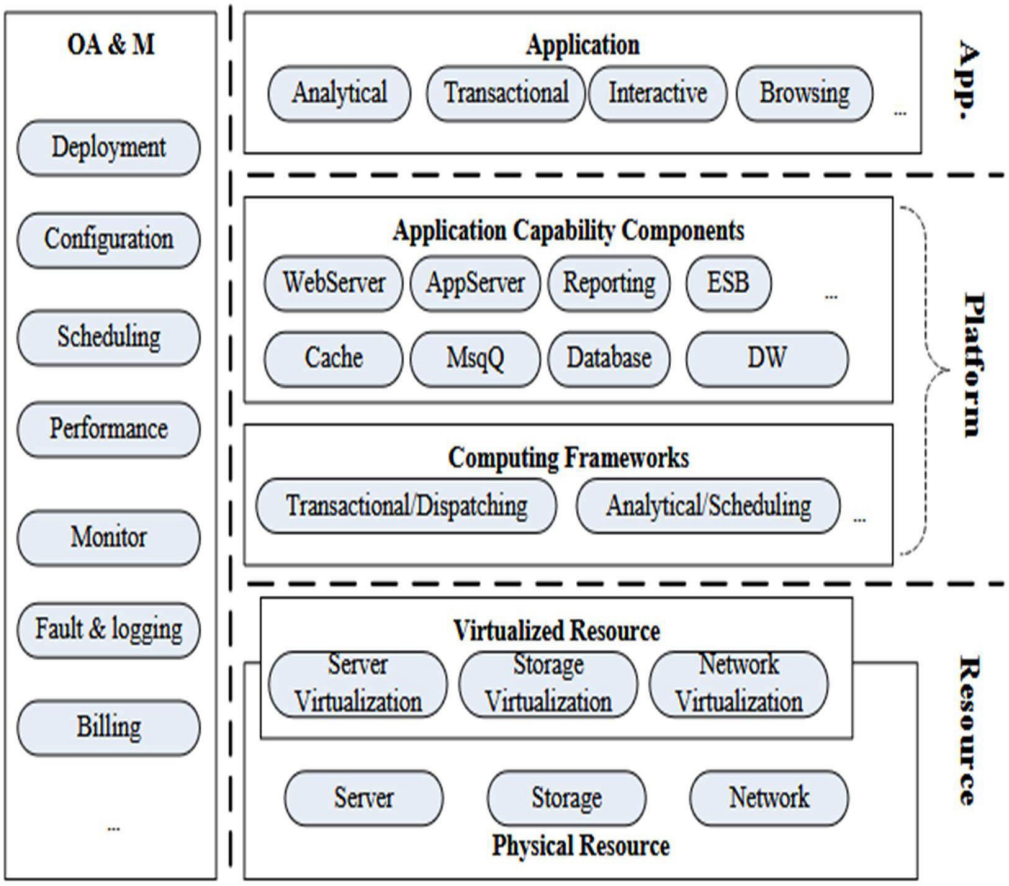
The application server and other components support the same general application logic as before with either on-demand capability or flexible management, such that no components will be the bottle neck of the whole system.This definition links the concept of services (IaaS/PaaS/SaaS)[1]

2.2.1 IaaS (Infrastructure as a Service): Providing a cloud-based infrastructure that defines a virtual location, storage, and network. It uses an infrared electronic controller. Examples: Amazon EC2, Microsoft Azure Virtual Machines, Google Compute Engine.

2.2.2 PaaS (Platform as a Service): Providing an application development and operation platform with internet-based management and updates, allowing developers to focus on programming and integration. Examples: Google App Engine, Microsoft Azure Application Services, Heroku.

2.2.3 SaaS (Software as a Service): Providing online applications without formal infrastructure for managing personal data or the platform. Examples: Google Workspace, Microsoft 365, Salesforce.

Fig.1 The Reference architecture



2.3 Cloud Computing Categories:

There are diverse dimensions to classify cloud computing, two commonly used categories are: service boundary and service type.

From the service boundary's view, cloud computing can be classified as public cloud, private cloud and hybrid cloud. The public cloud refers to services provided to external parties. The enterprises build and operate private cloud for themselves. Hybrid cloud shares resources between public cloud and private cloud by a secure network. Virtual Private Cloud (VPC) services released by Google[8] and Amazon[9] are examples of Hybrid cloud.

* From the service type's view, cloud computing can be classified as Infrastructure as a Service (IaaS), Platform as a Service (PaaS) and Software as a Service (SaaS). SaaS provide services to end users, while IaaS and PaaS provide services to ISV and developers - leaving a margin for 3-party application developers**.[1]**

2.4 Advantages and Risks:

The cloud computing is a Win-Win strategy for the service provider and the service consumer. We summarize the advantages as below:

2.4.1. Satisfy business requirements**:**

on demand by resizing the resource occu-

pied by application to fulfill the changing the customer requirements.

2.4.2. Lower cost and energy-saving:

By making use of low cost PC, customer-ized low power consuming hardware and server virtualization, both

CAPEX and OPEX are decreased.

2.4.3. Improve the efficiency of resource management:

through dynamic re-source scheduling

However there are also some major challenges to be studied.the risks:

2.4.4 Privacy and security:

Customer has concerns on their privacy and data security than traditional hosting service.

2.4.5 The continuity of service:

It refers to the factors that may negatively affected the continuity of cloud computing such as Internet problems, power cut-off, service disruption and system bugs. Followed are some typical cases of such problems: In November 2007, RackSpace, Amazon's competitor, stopped its service for 3 hours because of power cut-off at its data

center; in June 2008, Google App Engine service broke off for 6 hours due

to some bugs of storage system; In March 2009, Microsoft Azure experienced 22 hours' out of service caused by OS system update. Currently, the

public cloud provider based on virtualization, defines the reliability of ser-

vice as 99.9% in SLA.

2.4.6 Service migration:

Currently, no regularity organization have reached the agreement on the standardization of cloud computing's external interface.

As a result, once a customer started to use the service of a cloud computing

provider, he is most likely to be locked by the provider, which lay the customer in unfavorable conditions.

2.5 Kubernetes: As cloud environments become more dynamic and container-based, efficient orchestration and management of services become essential. Kubernetes was introduced to automate the deployment, scaling, monitoring, and maintenance of containerized applications. It provides a reliable way to manage workloads across multiple nodes while ensuring availability and performancee

2.5.1 WHAT IS KUBERNETES?

Kubernetes is an open source container orchestration system. It manages containerized applications across multiple hosts for deploying, monitoring, and scaling containers. Originally created by Google, in March of 2016 it was donated to the Cloud Native Computing Foundation (CNCF).

Kubernetes, or "k8s" or "kube" for short, allows the user to declare the desired state of an application using concepts such as "deployments" and "services." For example, the user may specify that they want a deployment with three instances of a Tomcat

web application running. Kubernetes starts and then continuously

monitors containers, providing auto-restart, re-scheduling, and

replication to ensure the application stays in the desired state.[2]

2.5.2 KEY KUBERNETES CONCEPTS:

Kubernetes resources can be created directly on the command line but are usually specified using Yet Another Markup Language(YAML). The available resources and the fields for each resource may change with new Kubernetes versions, so it's important to double-check the API reference for your version to know what's available. It's also important to use the correct "apiVersion" that matches your version of Kubernetes. This Refcard uses the API from Kubernetes 1.12, released 27 September 2018.

POD:

A Pod is a group of one or more containers. Kubernetes will schedule all containers for a pod into the same host, with the same network namespace, so they all have the same IP address and can access each other using localhost.

2.5.3 Cluster Structure: Consists of control planes and worker nodes. Key Components: Pods (basic container units), Deployments (deployment and update management), Services (application access interface), ConfigMaps and Secrets (control of settings and sensitive data). Operation: kube-apiserver, etcd as a central database, kube-scheduler for node selection, controllers for managing desired state, kubelet on nodes, kube-proxy for network management. Deployment Management: Via YAML/JSON definitions and kubectl as a controller. Scaling and Recovery: Supports autoscaling, automatic reconfiguration, and load distribution via Services and Ingress.

For Security and Compliance: Access Control: RBAC and resource permissions.

Network Policies: Network policies to restrict movement between containers.

Container Security: Pod Security Standards, image scanning, and security updates.

Secret Handling: Secrets and their encryption during transmission and storage.

Cryptography and Identity: TLS for components and necessary component authentication.

2.5.4 most famous versions of ks:

2.5.4.1 K8s

The most comprehensive platform for managing and orchestrating application containers across multiple environments.

It orchestrates the automated deployment, scaling, updating, and maintenance of containers.

2.5.4.2 K9s

An interactive CLI for managing Kubernetes clusters more dynamically and easily from the command line.

it displays resources and allows for quick navigation between them, with fast commands for accessing and updating.[3]

2.5.4.3 K3s

A lightweight Kubernetes distribution designed for resource-constrained environments like Edge and on-premises development.

It provides a complete Kubernetes experience with a simpler installation option and less complex configuration, using SQLite by default and a swappable database.[4]

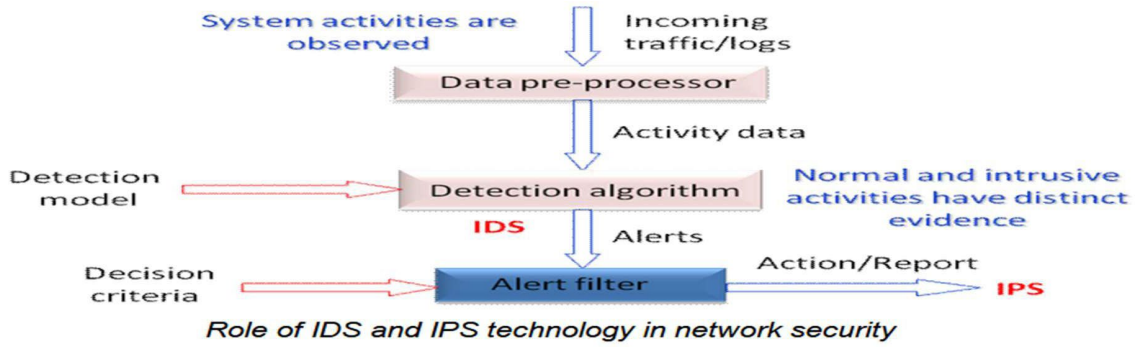
2.6 IDS:

IDS or Intrusion Detection Systems, are security components that aim to detect unusual and unauthorized activities within an organization's networks and systems and provide real-time alerts to enable rapid response. IDS typically operate in two main models: network-based IDS that monitors network traffic for known or suspected patterns, and host-based IDS that monitors system behavior and changes at the device level (files, processes, user permissions).

Detection mechanisms generally rely on two approaches: signature-based, which recognizes predefined attack patterns, and anomaly-based, which looks for behavior that deviates from the normal baseline. IDS operates within the organization's external protection layer as part of the cybersecurity stack including SIEM and SOC, and is used to generate alerts, analyze incidents, and document response. In cloud environments, examples include AWS GuardDuty, Azure Defender for Cloud, and Google Cloud IDS, which offer network- or host-based detection capabilities and integrate with reporting and investigation tools.

Key advantages include improved incident response, reduced recovery time, and enhanced security visibility, while facing limitations such as the need for careful policy configuration, alert-volume management, and potential gaps in advanced or complex attacks.[5]

Figure 2 How ids works:



2.6.1 Types of IDS:

- Basic Types Network-Based IDS (NIDS): Monitors traffic across networks and interfaces.
- Host-Hosted IDS (HIDS): Monitors the activity of the system itself (files, logs, processes).
- Integrated IDS/IPS: Combines detection, response, and prevention capabilities into a single solution.

2.6.2 Mechanism of IDS:

- Monitoring: Compares real-time data with known safe models and attacks.
- Analysis: Uses signature-based prediction, machine learning, and anomaly-based behavior analysis.
- Alert and Response: Sends notifications and logs, and helps inform intrusion response policies.

Common examples: Network-based IDS (NIDS) and Hosted IDS (HIDS), as well as IDS/IPS systems integrated into some solutions.

2.6.3 Importance in Cloud:

Environments Protecting the access and data layer across SaaS/PaaS/IaaS services and applications. Detecting unauthorized activity across multiple pathways (virtual networks, containers, applications). Enhancing compliance and governance through security logging and rapid remediation solutions. Adapting to evolving threats and ensuring operational flexibility across multi-region deployments.

2.7 Falco:

Falco is an intrusion detection and tampering prevention system that, at its core, relies on monitoring system events at the kernel level. It uses kernel instrumentation techniques to intercept in the Linux kernel via kernel probes (both kprobes in older versions and eBPF in newer ones) to capture system events such as file actions, network connections, and process creation/termination, and streams them to the user-space processing layer where Falco's rule set, written in its own rule language, is applied to identify unusual or suspicious behavioral patterns. Falco acts as an intermediary layer that separates event observation from interpretation; it aggregates data from the event source (the kernel) and applies rule logic to generate alerts, with the capability to configure immediate responses or drive investigations. The design enhances performance by using kernel-tunneling mechanisms embedded in the kernel (such as eBPF) to reduce resource consumption, while keeping rule interpretation in user space for flexibility in development and updates. It also supports linking alerts to other security systems and providing comprehensive reports and visibility via APIs and centralized services. Effective deployment relies on finely tuned rules and continuous threat intelligence updates aligned with the operating environment and configurations, with flexibility to operate in cloud environments and across various Linux distributions, and integration with incident management and auditing tools.[6]

2.7.1 How it works :

Runtime monitoring: Operates on top of the Kutns kernel layer via an event-listening mechanism, covering the behavior of containers and their hosts.

Customizable rules: Uses a set of (warning) rules that detect unusual behavior such as unauthorized file modifications or out-of-band operations.

Alert technologies: Records alerts and sends them to SIEM systems or other alerting tools, facilitating automated response.

Integration and extension: Seamlessly integrates with Kubernetes and DevOps processes, and supports extracting examples from MITRE ATT&CK in analysis environments.

2.7.2 Falco Rules and Principles Rule Format:

Expresses a specific event (opening a sensitive file or executing an unauthorized command) with time or contextual conditions. Rule Management: Built-in default rules can be run and modified, and custom rules can be added to suit your environment. Implementation: Operates as a proxy element (Falco daemon) or across multiple blocks in large environments.[6]

2.7.3 What sets it apart from other detection systems:

Runtime focus: It monitors behavior in the real-time environment, not just for post-processing analysis.

Container and hybrid system support: Designed to work efficiently with Kubernetes, containers, and Linux systems.

Easy customization: Customizable rules and deep integration with SIEM and SOAR tools. Not just signature-based: It enhances inference with real-time behavior and patterns.

2.8 File browser:

File Browser, also known as File Explorer, is a user interface tool that lets users browse and organize files and folders stored on a computer or within a cloud computing environment. It can operate as a graphical or text-based interface that enables sorting, searching, opening, moving, copying, and deleting files, while providing detailed information such as access permissions, file sizes, and creation/modification dates. Types of File Browser vary by environment; desktop browsers are common in operating systems like Windows, macOS, and Linux, in addition to built-in browsers within servers' file management systems and web applications, and sometimes it integrates with cloud functions such as displaying storage content from AWS S3 or Google Cloud Storage. The experience relies on a usability-focused design that facilitates navigation of the hierarchical folder tree, with immediate options for search, filtering, and multi-select, as well as file management capabilities like version awareness, permission control, and integration with collaborative tools. In security and governance environments, a File Browser can include fine-grained access controls, secure transfer protocols, user

activity tracking, and possibly certificate embedding and data encryption settings to protect content during browsing and transfer. If you'd like a specialized clarification (e.g., Linux GUI browsers or cloud-based browsers) or a comparison among several solutions

2.8.1 How it works:

Background storage: Relies on a cloud provider that stores files as objects with specific paths and access links. Viewing interface: Provides a graphical interface similar to a file explorer for browsing and interacting with folders and files. Permissions and sharing: Supports access permissions, link sharing, and access control mechanisms. Basic operations: Upload, download, create folders, delete, rename, search, and move.

2.8.2 Popular Features:

Easily view and organize files using simple folder structures. Drag and drop functionality, and batch uploads. Access control and encryption during transfer and storage. Quick search and filtering by type, size, and date. Integration with other cloud tools (SIEM, CI/CD, backup systems).

2.8.3 The security service that provides a cloud file browser (cloud file browser):

1-Access and Identity Control User authentication on sessions Precise deployment across roles and permissions on resources Multi-Factor Authentication (MFA) support .

2-Data Encryption and Protection Teleread-in-Transit (TLS) Teleread-in-Atom (AES/Other depending on the provider) Key management via KMS services.

3-Trace and Persistence Log of Japanese user actions (audit logs) Access, change, and download reports Support for core standards (e.g., GDPR, HIPAA depending on context)

4-Tracking and Compliance User activity and process logging (Audit logs) Access, change, and download reports Compliance standards support (e.g., GDPR, HIPAA as contextual)

5-Link and Temporary Access Management Pre-signed URLs for sharing control Modified and trackable sharing permissions 6-Security and Configuration Policies Bucket/Object policies Compliance checks and configuration checks Alerts for unusual activity or unauthorized access.

7-Tamper and Data Loss Protection Versioning support for restoring previous versions Protection against accidental deletion (Object lock/immutability on some providers) 8-Network Integrity and Security Isolating networks and applications between S3/GCS/Azure environments Access control via network standards (VPC endpoints, firewall rules)

2.9 Conclusion:

This section covers several important terms, including: basic definitions of cloud and its IaaS/PaaS/SaaS services; deployment models; Kubernetes fundamentals with examples of the K9s tool and the K3s option; Falco and IDS systems; and security in cloud environments. It also discusses potential examples and comparisons, as well as source options for references and documentation in addition of file browser and how it works

CHAPTER 3

proposed system

3.1 Intro:

This section presents cloud environment simulation models with the addition of a FALCO protocol detection system for monitoring events and attacks, along with the selection of appropriate tools for it. The background is based on working on the system before and after security events in the environment.

work flow and project phase:

3.2 Choosing the Work Environment and Operating System

A virtual machine (VM) was chosen to achieve complete environmental isolation between the development, testing, and production environments. This also allows for rapid replication and backup, and isolated experimentation without impacting other systems. The VM platform is used because it offers scalability, flexible resource management, and maintenance

For the operating system, Ubuntu 64-bit was chosen for its open framework, ease of management, availability of modern packages, and extensive support for security libraries, data analysis tools, and live data streams. Ubuntu 64-bit enables seamless integration with containers and cloud services, provides an active support community, and offers regular security updates, thus enhancing the stability of the proposed system and its compliance with security standards. Choosing this environment supports rapid development, compatibility with monitoring and auditing tools, and streamlined deployment and update processes throughout the development phases

3.3 Cloud Environment Component :

Selection Kubernetes was chosen as the management framework for coordinating and managing system components, with a lightweight distribution like K3S used to simulate a cloud environment with limited resources and simplify node management and development. Two virtual models were designed to represent the cloud architecture:

a master node representing the central server for service management and incident detection.

worker node representing the cloud client where containers run and traffic is routed. This configuration enables realistic simulation of load distribution, resource management, and the enforcement of security and compliance policies within resource constraints, with the ability to scale incrementally as the project evolves. The system also supports reduced configuration and testing time, as well as enhanced operational stability through task isolation and easy service migration between nodes.

3.4 Choosing and downloading a cloud-based intrusion detection system:

Falco was selected as our Cloud Intrusion Detection (IDS) solution due to its robust monitoring capabilities in sensitive cloud environments and its ease of adaptation to reconfigurable infrastructure. Falco's functionality was thoroughly analyzed; it relies on monitoring system events and anomalous behavior through updatable behavioral rules and offers seamless integration with container platforms and cloud service applications. Falco's core rules and their respective tiers were analyzed and documented, and a policy framework was developed that addresses security and compliance requirements, enabling the implementation of real-time notifications and the generation of audit reports. Based on this analysis, we concluded that choosing Falco contributes to reduced intrusion response times, improved security visibility, and a scalable and updatable mechanism that adapts to the evolving cloud environment and threat landscape.

3.5 Downloading the Business Performance Measurement Tools

(k6 and stress-ng) :

The k6 tool was selected as an open-source load measurement tool to test the performance of APIs and cloud services with realistic response time reports and request flows, while stress-ng is used to measure system stability and resource tolerance by generating system-wide composite loads and evaluating their impact on responsiveness and security.

Both tools will be built in two phases:

1-the first before a sufficient number of security events occur on the Falco system to measure baseline performance and analyze scope.

2-the second after security events occur to deepen understanding of the impact of limitations and threats on detection efficiency and response time reduction.

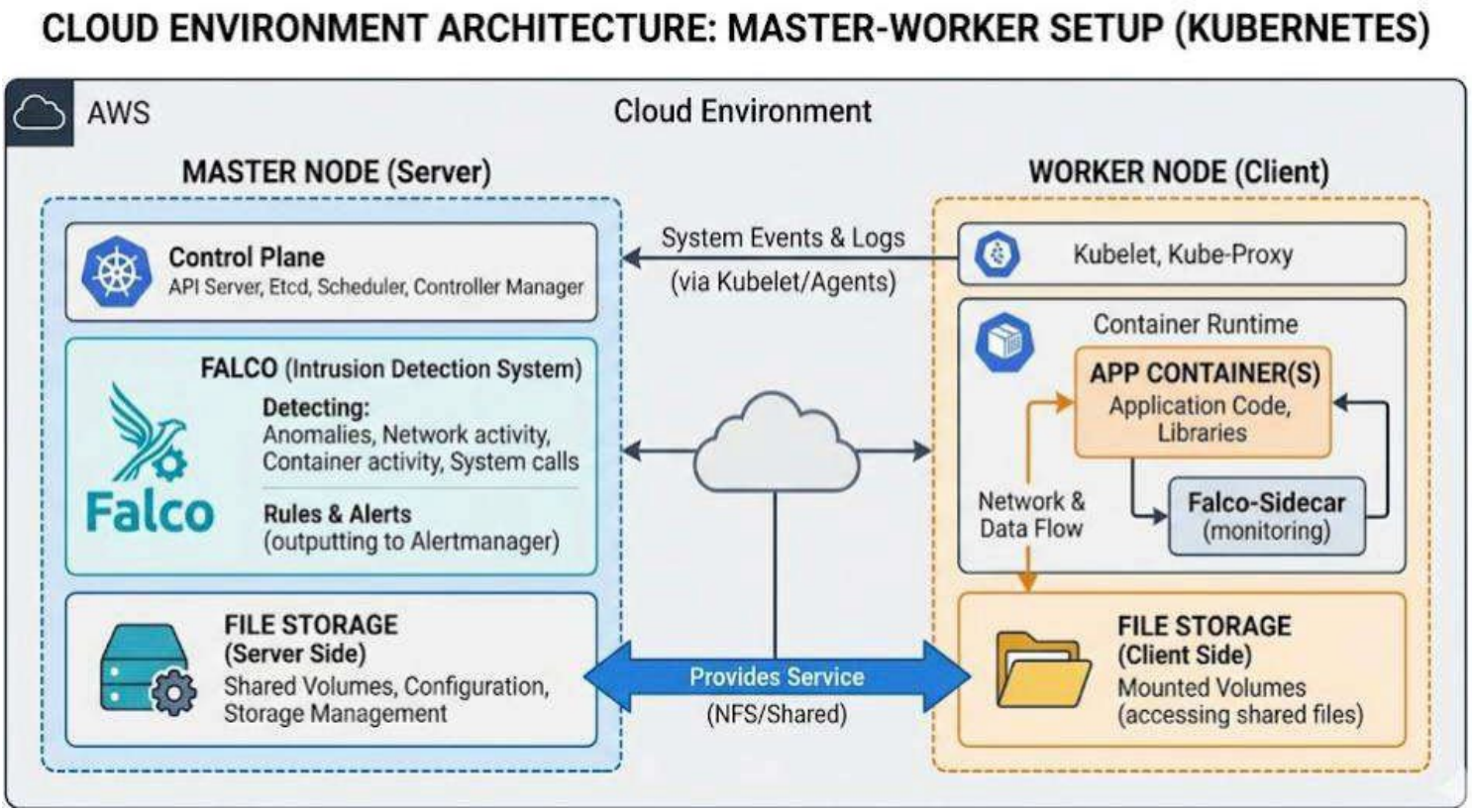
The use of these two tools aims to provide comparable measurements that help assess the effectiveness of an intrusion detection system when cloud services operate alongside real loads and under multiple security pressures.

3.6 Cloud file storage:

This project offers a cloud storage service with various add-ons and file uploads via a dedicated interface. Files are stored in a secure, encrypted database before being saved. The core process involves translating files before saving them and storing them as binary assets with metadata for precise optimization

(owner, time period, file type, etc.). This ensures content protection even in the event of a system breach.
This design aims to support ease of retrieval and searching while maintaining regulatory compliance and providing a framework that can be studied in the project in terms of performance, security and reliability.

the proposed cloud architecture used in this project. The environment consists of a K3s cluster with a master node responsible for management and intrusion detection using Falco, while the worker node hosts the cloud storage service and client-side operations. This setup was selected to simulate a lightweight cloud environment and evaluate IDS behavior under realistic conditions



As shown in the architecture, the separation between the master and worker nodes improves monitoring and simplifies resource management. Integrating Falco with the control layer enabled centralized event observation while maintaining service availability on the worker side

3.7 Conclusion:

The project was built in a virtual environment based on the 64-bit Ubuntu operating system, where the K3s container was chosen as a suitable type for running services within a cloud architecture and facilitating deployment and management with limited resources. Subsequently, a cloud intrusion detection system and workload measurement tools were installed on the server to monitor anomalies and accurately analyze performance in two consecutive phases: the first before any security events or breaches are detected by the IDS systems, and the second simultaneously with the occurrence of such events and the application of cloud storage services.
This allows for measuring the impact of security oversight on service stability and performance efficiency. In the first phase, the system focuses on establishing monitoring lines, configuring threat prediction rules, and ensuring the readiness of transmission channels and APIs. The second phase aims to provide real-time, integrated monitoring with continuous operation, thereby enhancing risk detection and mitigation during upload and storage traffic on the cloud storage service. The design relies on integrating a scalable container layer with advanced monitoring mechanisms, enabling horizontal container expansion and integration with the pre-stored encrypted storage system. Furthermore, it ensures workload measurement and performance evaluation systems that support compliance and security verification over time.

CHAPTER 4

Implementation

4.1 intro

In this chapter, we present the practical implementation of our proposed system. The process consists

of the following key stages:

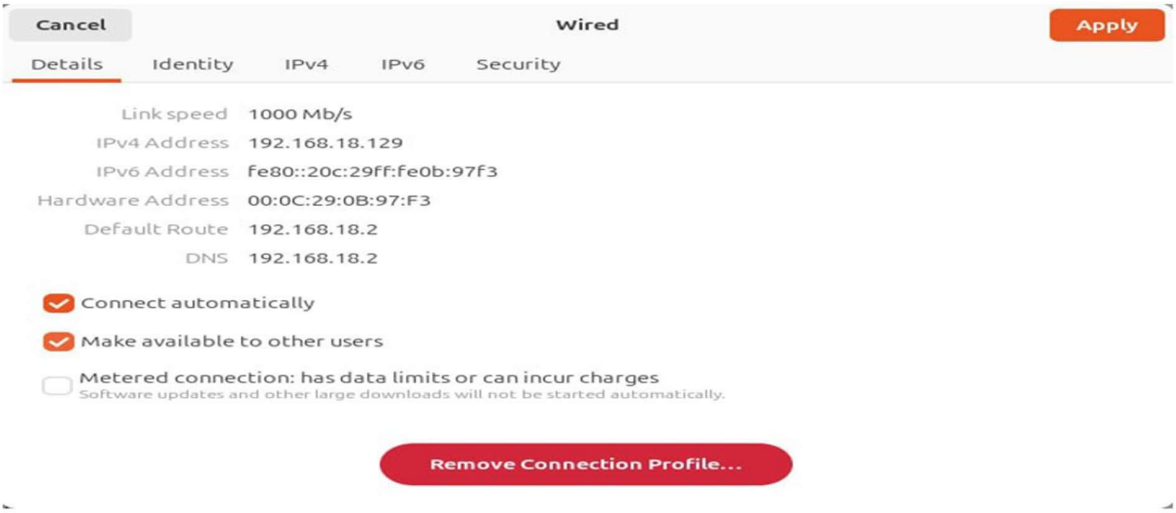
4.2 Two VMs have been created, and a 64-bit Ubuntu operating system image has been downloaded on each VM.

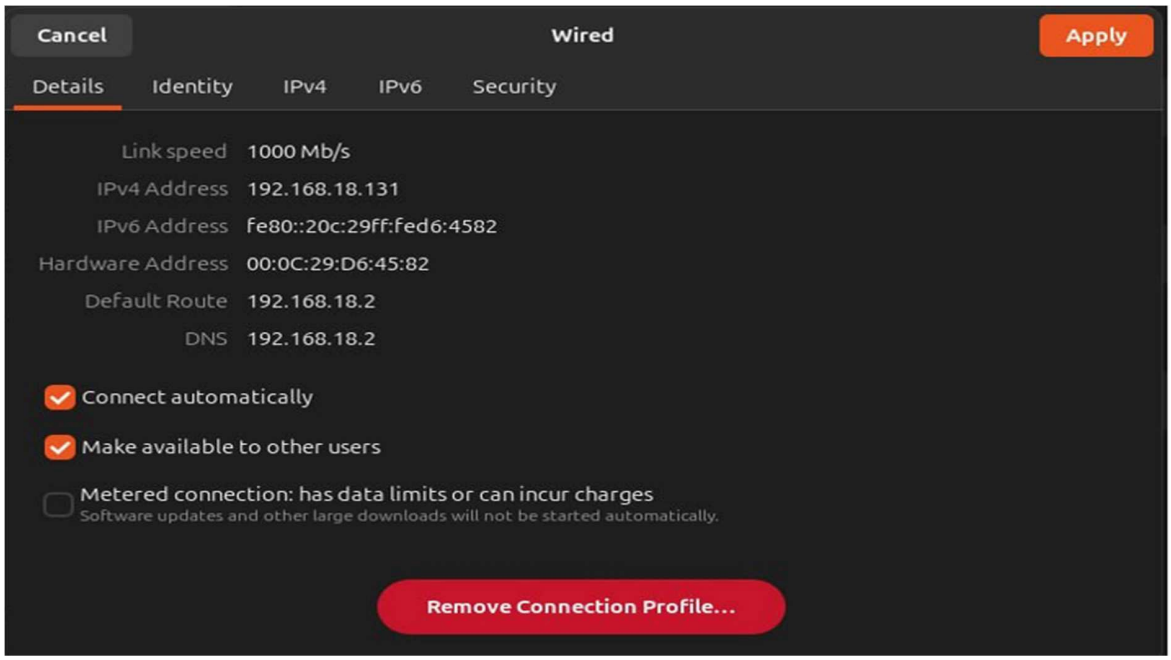
4.3 Container images for K3s have been downloaded on both virtual machines, and the master node is considered the server node, while the worker node is the client node.

```
● k3s.service - Lightweight Kubernetes
     Loaded: loaded (/etc/systemd/system/k3s.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-04-24 14:10:02 CEST; 1 day 8h ago
       Docs: https://k3s.io
   Main PID: 1403 (k3s-server)
      Tasks: 113
     Memory: 783.9M (peak: 1.0G swap: 5.5M swap peak: 5.5M)
        CPU: 58min 25.086s
     CGroup: /system.slice/k3s.service
             ├─1403 "/usr/local/bin/k3s server"
             ├─1850 "containerd "
             ├─2825 /var/lib/rancher/k3s/data/d76b59b2203578bb4cb3438338ccad2f0>
             ├─2844 /var/lib/rancher/k3s/data/d76b59b2203578bb4cb3438338ccad2f0>
             ├─2976 /var/lib/rancher/k3s/data/d76b59b2203578bb4cb3438338ccad2f0>
             ├─3018 /var/lib/rancher/k3s/data/d76b59b2203578bb4cb3438338ccad2f0>
             └─3065 /var/lib/rancher/k3s/data/d76b59b2203578bb4cb3438338ccad2f0>
```

```
● k3s-agent.service - Lightweight Kubernetes
     Loaded: loaded (/etc/systemd/system/k3s-agent.service; enabled; preset: enabl>
     Active: active (running) since Fri 2026-04-24 14:46:20 CEST; 8s ago
       Docs: https://k3s.io
    Process: 4612 ExecStartPre=/sbin/modprobe br_netfilter (code=exited, status=0/>
    Process: 4615 ExecStartPre=/sbin/modprobe overlay (code=exited, status=0/SUCCE>
   Main PID: 4617 (k3s-agent)
      Tasks: 33
     Memory: 67.3M (peak: 72.2M)
        CPU: 4.782s
     CGroup: /system.slice/k3s-agent.service
             ├─4617 "/usr/local/bin/k3s agent"
             ├─4634 "containerd "
             └─5034 /var/lib/rancher/k3s/data/d76b59b2203578bb4cb3438338ccad2f0d3e>
```

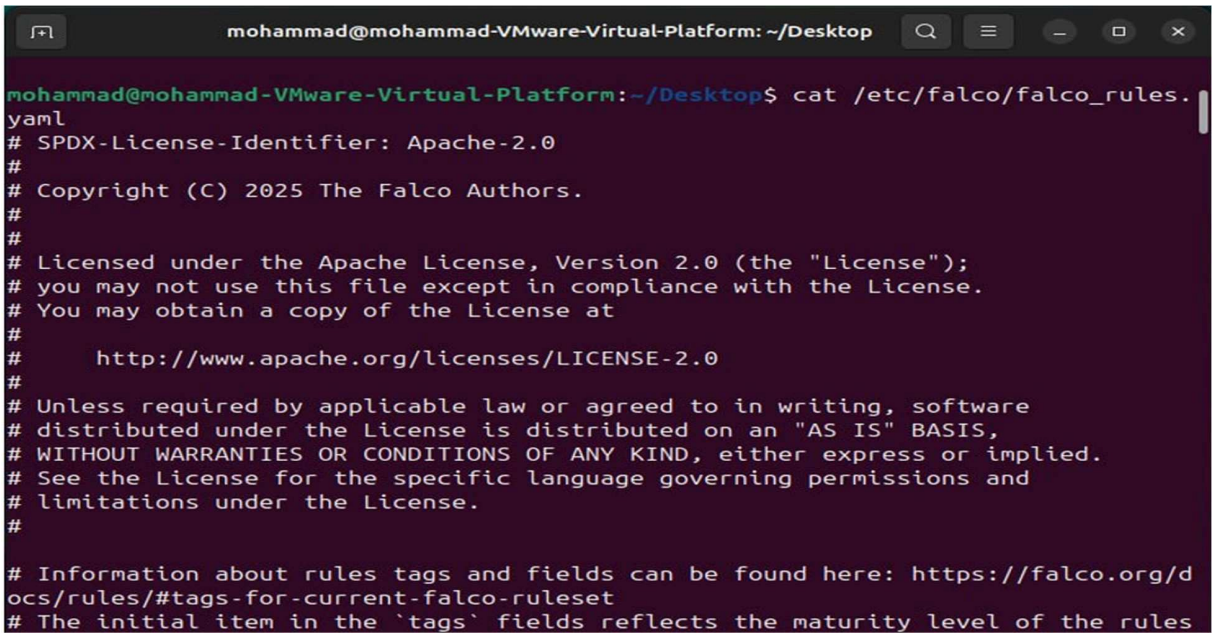
4.4 IPS for master and worker nodes:





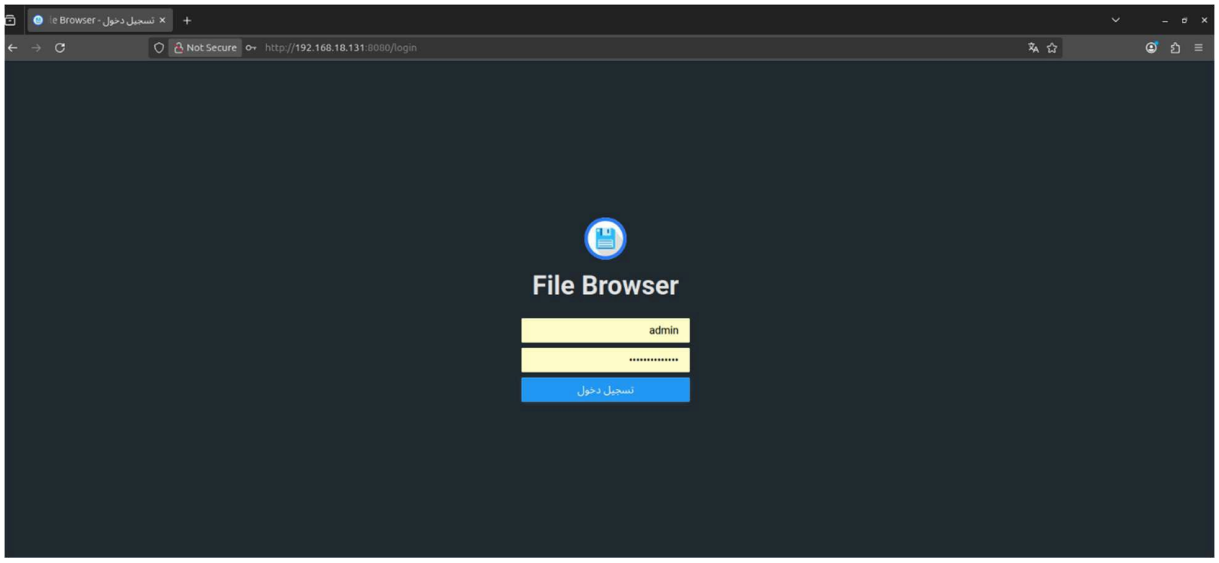
4.5 Falco intrusion detection system, along with its rule file and configuration, has been downloaded on the cloud server or on the master node.

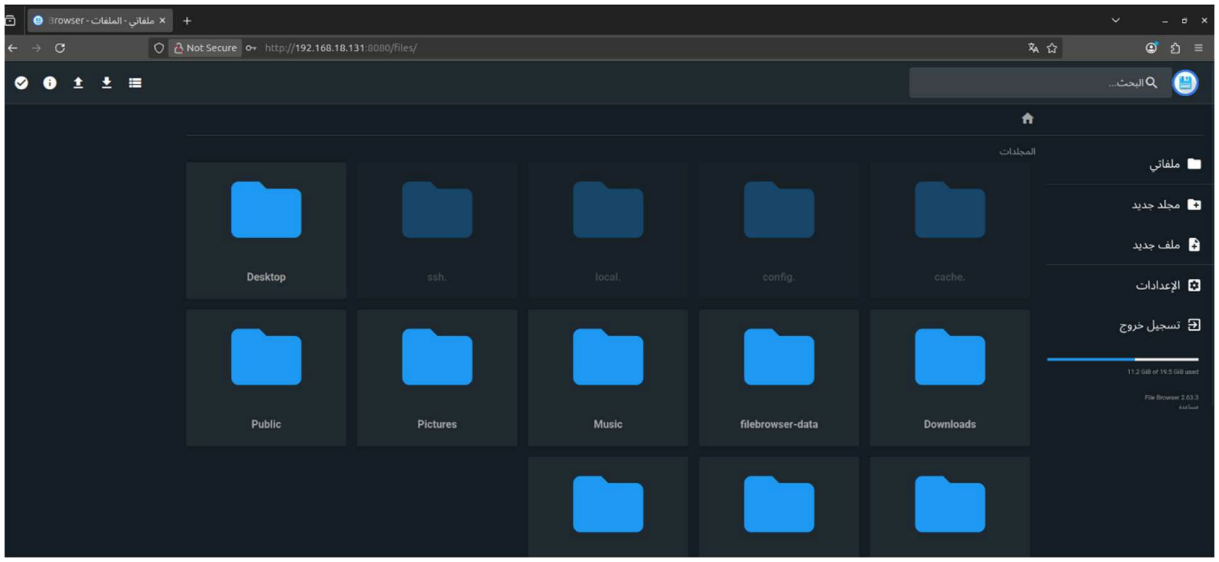
4.6 FALCO rules:



4.7 Choosing, downloading, and enabling a cloud file storage service to offer it as a service from the cloud server to the client.

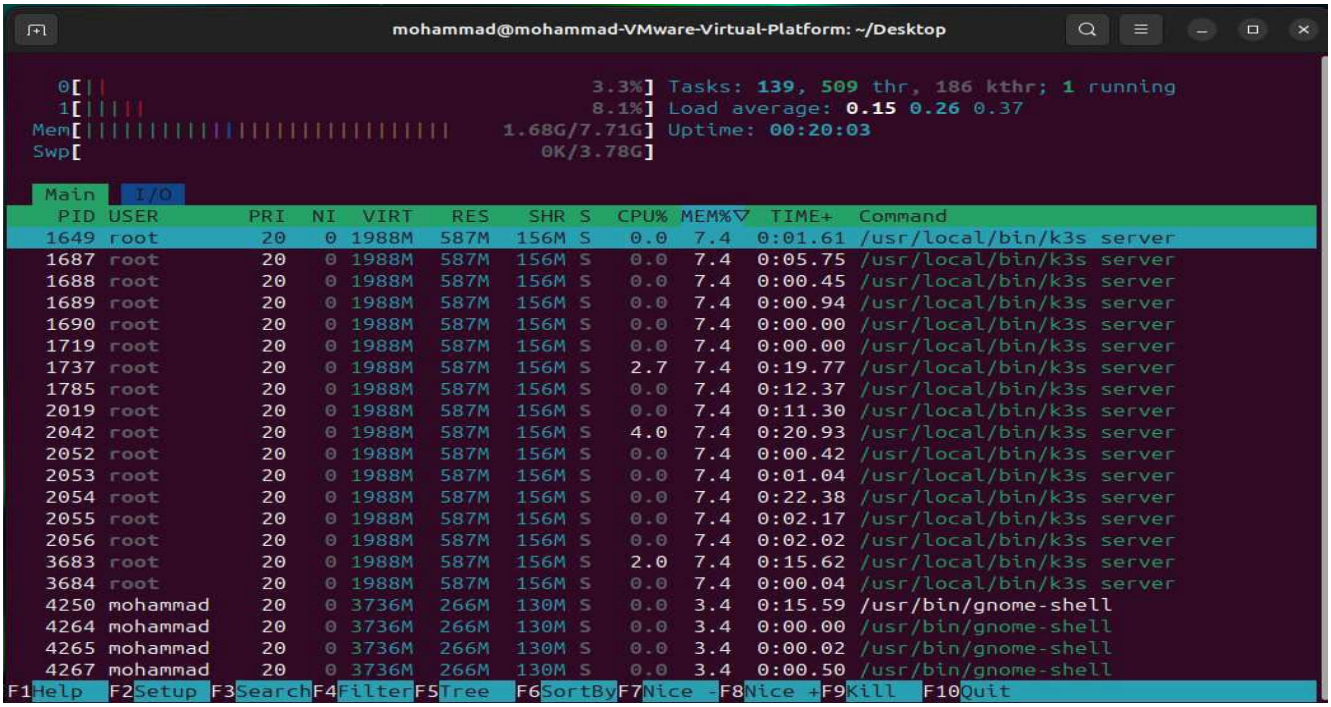
```
mohammad@mohammad-VMware-Virtual-Platform:~/Desktop$ sudo kubectl get pods -o wide
[sudo] password for mohammad:
NAME                          READY   STATUS    RESTARTS       AGE   IP           NODE
 NOMINATED NODE   READINESS GATES
filebrowser-dfffdff5-dkn99    1/1     Running   1 (138m ago)   34h   10.42.1.7    worker1
 <none>           <none>
```

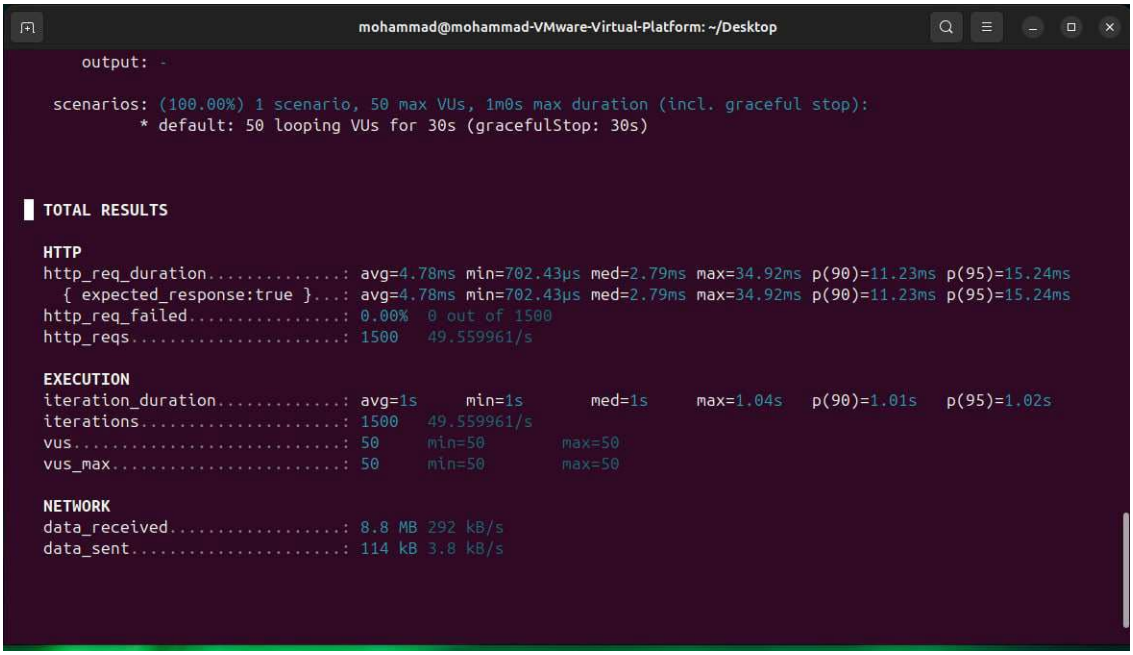




4.8 Load testing using my tools k6 and stress-ng before

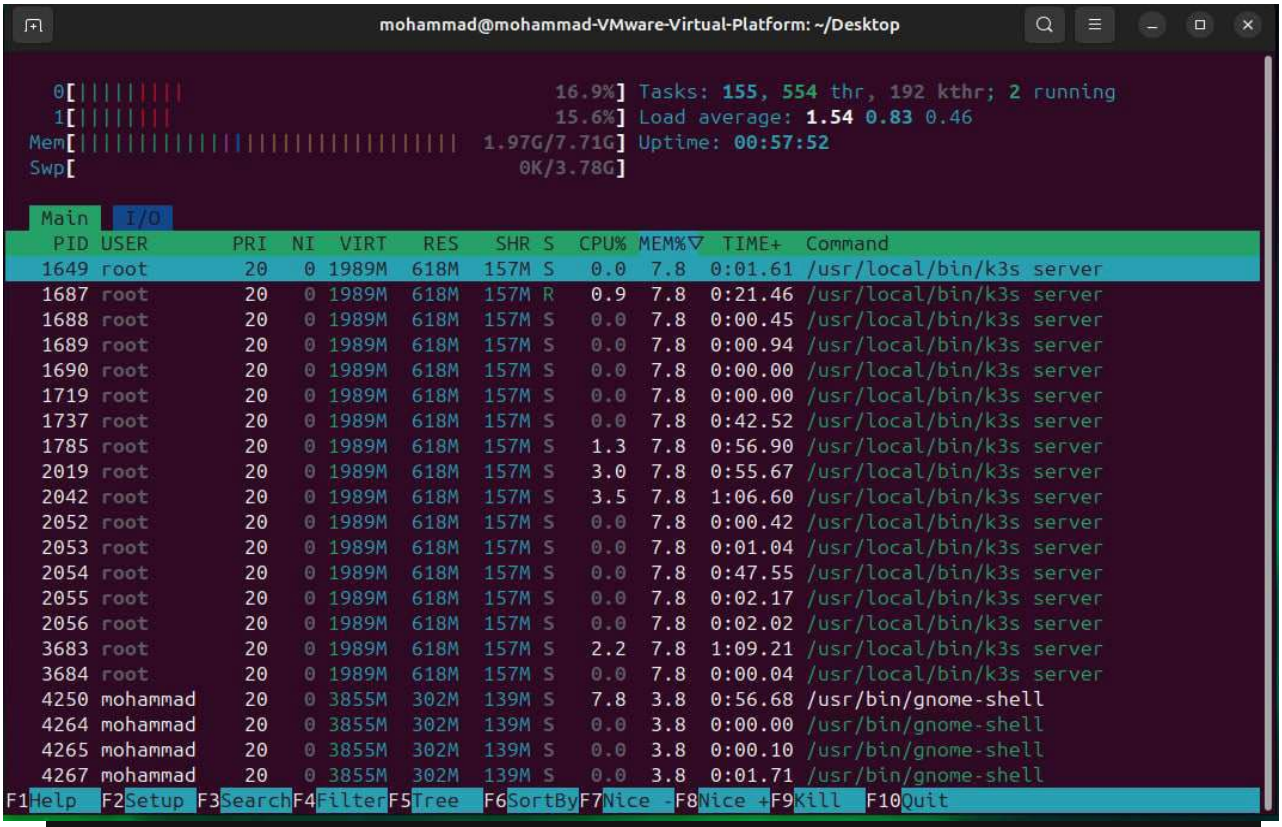
carrying out an attack on the cloud server .

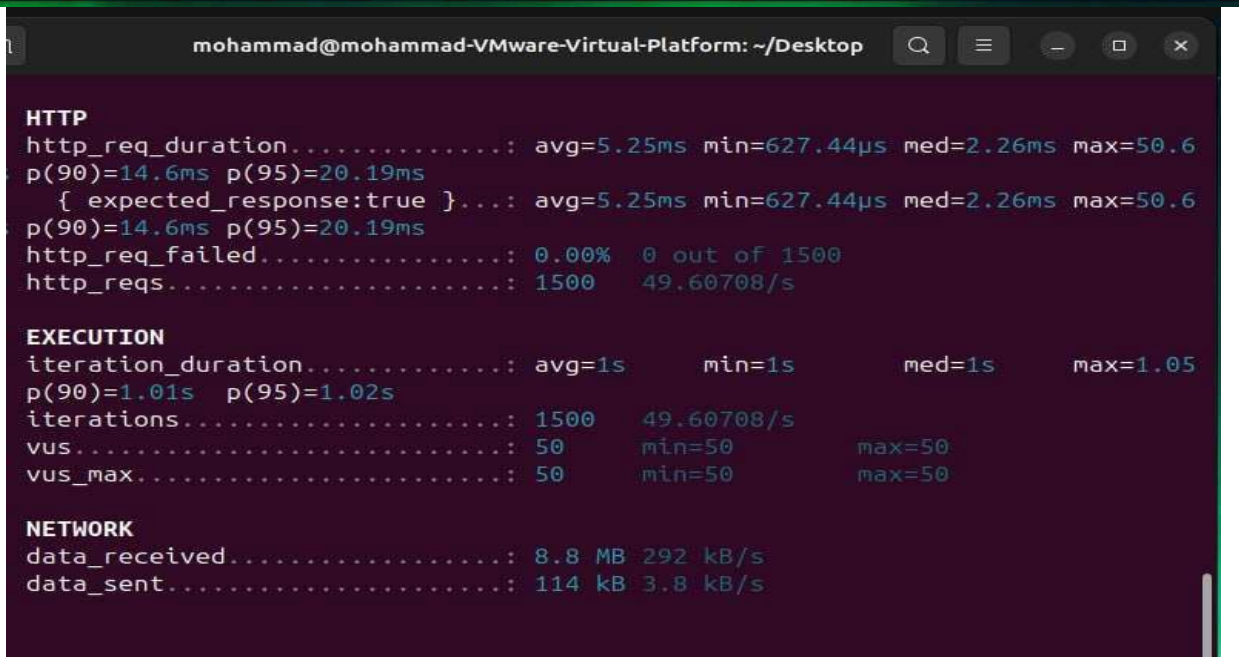




4.9 Load testing using my tools k6 and stress-ng after

carrying out an attack on the cloud server





Tab.2

	Before	After
CPU Usage	3% – 8%	15% – 17%
Memory Usage	1.68 GB	1.97 GB
Load Average	0.15	1.54
Response Time (avg)	4.78 ms	5.25 ms
Max Response Time	34.9 ms	50.6 ms
Requests/sec	~49.5 req/s	~49.6 req/s
Failed Requests	0%	0%
Attack Detection	❌ غير موجود	✅ موجود (Falco)

The results indicate that enabling Falco increased CPU and memory utilization slightly while maintaining acceptable response time and zero failed requests, demonstrating that security monitoring introduced limited overhead.

4.10 conclusion:

In this chapter, we successfully implemented and validated the full pipeline of our proposed system.

From building the cloud environment and the ids on cloud server in addition of enabling the file storage and built a successful load test on the mater node at al

CHAPTER 5

conclusion

5.1 intro

This section reviews the findings from the cloud storage service project, where the results showed that the presence of the Falcon threat detection system enhances data protection and compliance, while having a limited impact on performance in cases of security events occurring concurrently with monitoring and analysis operations.

5.2 SUMMER OF RESULTS:

The project's findings summary shows that the presence of cloud intrusion detection systems has a relative impact on cloud service performance during security incidents. According to the outputs of previously used workload measurement tools, processing and response speeds decrease during threat periods as the workload from monitoring, analysis, and threat prediction increases, potentially leading to missed incidents or a temporary decrease in recovery accuracy.

References:

1. Mell, P., & Grance, T. (2011). *The NIST Definition of Cloud Computing (Special Publication 800-145).* National Institute of Standards and Technology (NIST).
2. Burns, B., Beda, J., & Hightower, K. (2019). *Kubernetes: Up and Running: Dive into the Future of Infrastructure* (2nd ed.). O'Reilly Media.
3. The Kubernetes Authors. *Kubernetes Documentation.* Available at: https://kubernetes.io/docs/
4. Rancher Labs. *K3s Lightweight Kubernetes Documentation.* Available at: https://docs.k3s.io/
5. Scarfone, K., & Mell, P. (2007). *Guide to Intrusion Detection and Prevention Systems (IDPS), NIST Special Publication 800-94.*
6. Falco Project. *Falco Documentation – Cloud Native Runtime Security.* Available at: https://falco.org/docs/
7. Sysdig. *Runtime Threat Detection with Falco.* Available at: https://sysdig.com/opensource/falco/
8. Grafana Labs. *k6 Load Testing Documentation.* Available at: https://k6.io/docs/
9. Collet, C. *stress-ng Manual and Documentation.* Available at: https://manpages.ubuntu.com/
10. File Browser Project. *Official File Browser Documentation.* Available at: https://filebrowser.org/
11. Amazon Web Services (AWS). *Overview of Cloud Computing Services.* Available at: https://aws.amazon.com/
12. Google Cloud. *Cloud Architecture Framework.* Available at: https://cloud.google.com/architecture
13. Microsoft Azure. *Azure Architecture Center.* Available at: https://learn.microsoft.com/azure/architecture/
14. CNCF (Cloud Native Computing Foundation). *Cloud Native Landscape and Kubernetes Overview.* Available at: https://www.cncf.io/
15. MITRE ATT&CK Framework. *Enterprise Techniques and Detection Guidance.* Available at: https://attack.mitre.org/